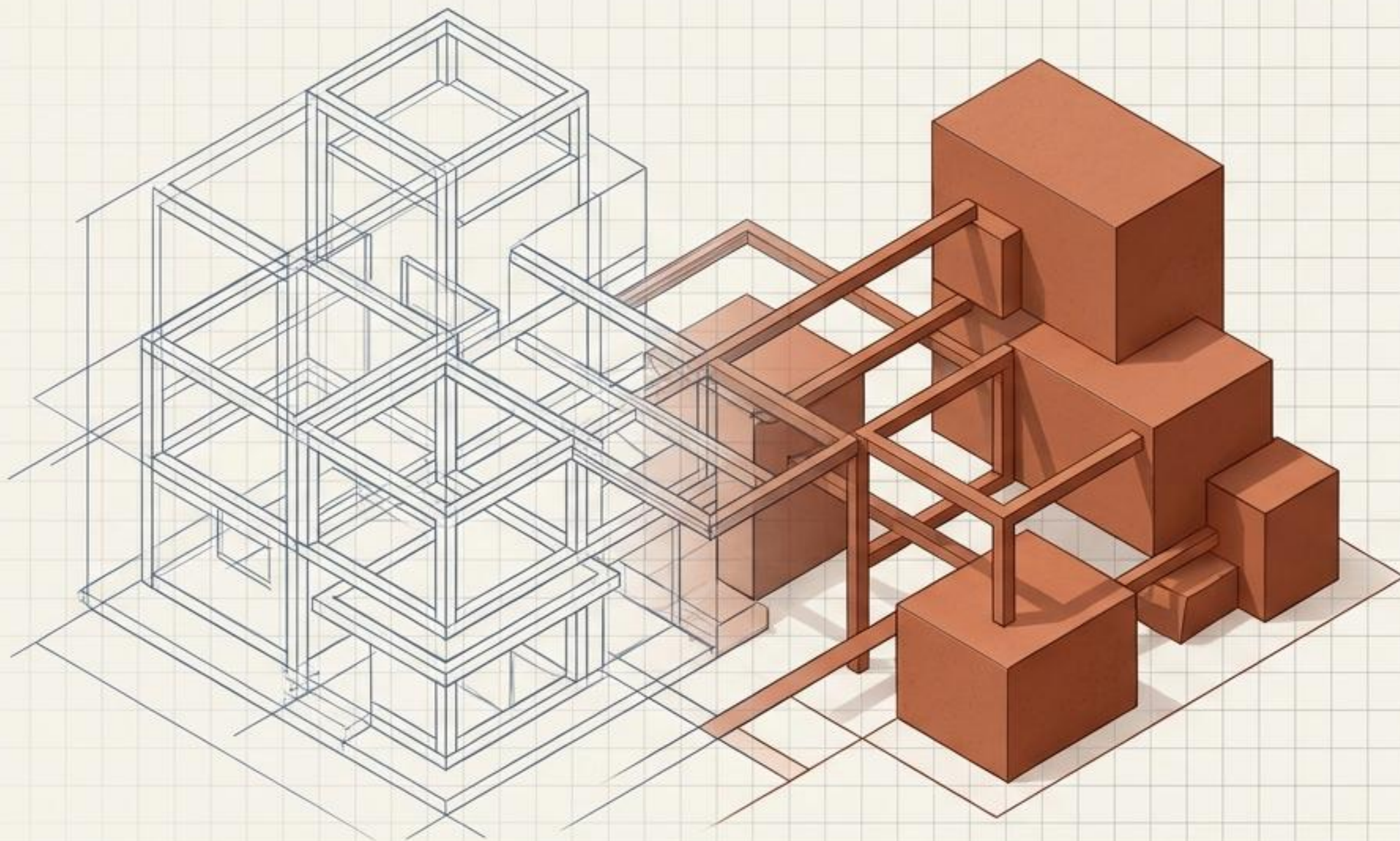




데이터에서 지식으로: 온톨로지와 의미 기반 AI

LLM의 한계를 극복하고 신뢰할 수 있는 RAG 시스템을 구축하기 위한 시맨틱 웹 기술 마스터클래스

해결해야 할 6가지 핵심 질문

- ☐ 1. 온톨로지는 무엇이고 지식그래프와 어떻게 다른가?
- ☐ 2. FOL, RDF, OWL, SHACL, SPARQL의 정확한 역할은 무엇인가?
- ☐ 3. TTL은 RDF 문법인가, OWL 문법인가?
- ☐ 4. @prefix는 외부 TBox를 가져오는 기능인가?
- ☐ 5. Protégé와 OWL Reasoner는 실무에서 어떤 역할을 하는가?
- ☐ 6. 추론(Reasoning), 질의(SPARQL), LLM(RAG)은 어떻게 결합되는가?

의미 기반 AI의 핵심 파이프라인



AI의 두 가지 흐름과 결합의 필요성

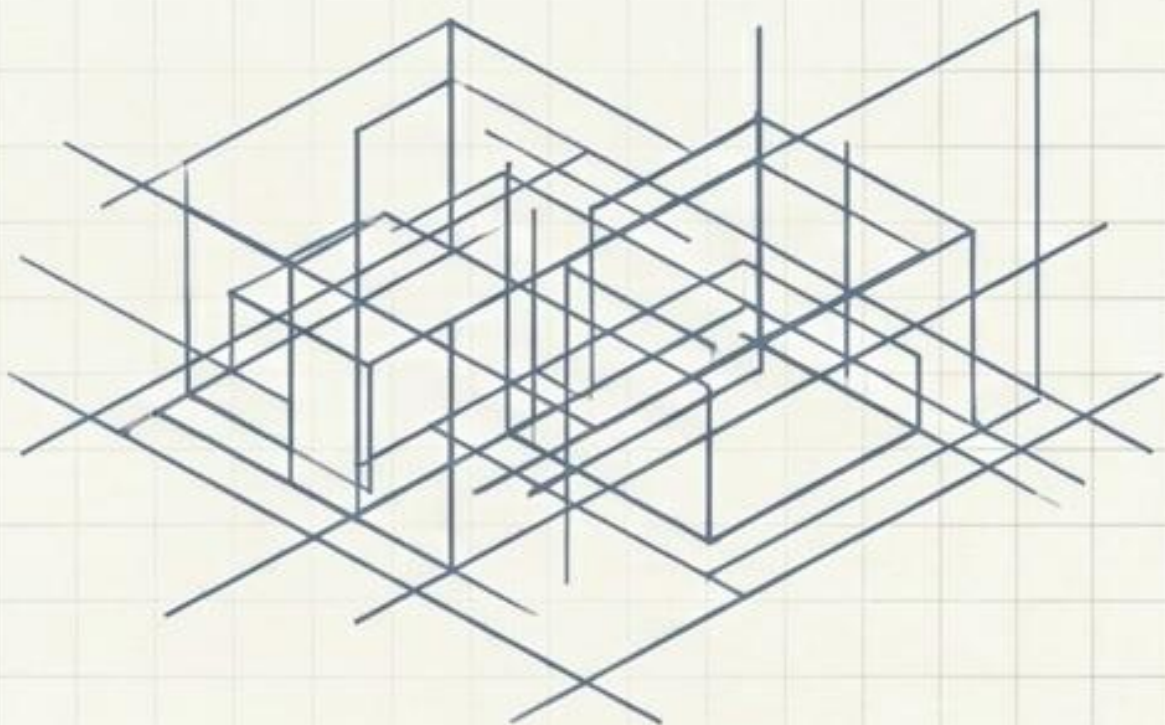
심볼릭 AI (Symbolic AI)	연결주의 AI (Connectionist AI / LLMs)
- 방식: 기호와 규칙 명시 (예: place(강남역). congested(강남역). advise(X, 방문 자제) :- congested(X).) - 장단점: 높은 설명력과 투명성 / 낮은 확장성 (사람이 직접 작성해야 함)	- 방식: 대량의 데이터에서 패턴 학습 (확률 기반 단어 예측) - 장단점: 강력한 유연성과 생성 능력 / 환각(Hallucination), 논리적 일관성 부족, 도메인 제약 유지 불가

온톨로지의 역할: LLM의 한계 극복



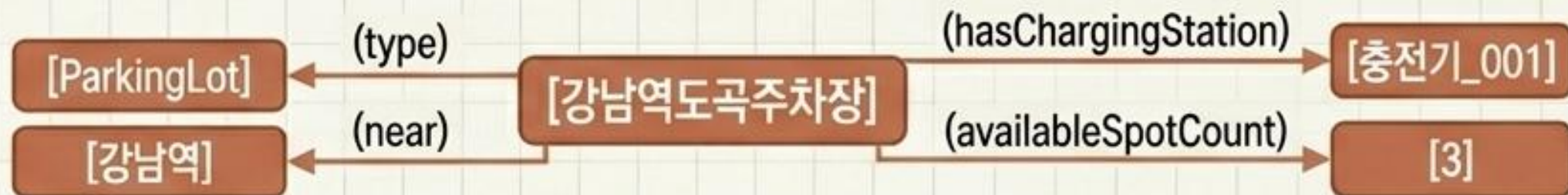
LLM은 뛰어난 언어 생성기이지만, 주차장, 충전소, 지하철역 간의 업무적 제약을 언어 패턴만으로 강제할 수 없습니다. 온톨로지는 개념, 관계, 제약을 명시적으로 정의하여 LLM에 논리적 뼈대를 제공합니다.

온톨로지 (Ontology)



비유: 건축 설계도 (문법책)
역할: 개념, 관계, 제약 정의
중심 질문: 무엇이 존재할 수 있는가?
성격: 의미 구조 (Semantic Structure)

온톨로지 예시:
ParkingLot은 Facility이다.



지식그래프 (Knowledge Graph)



비유: 설계도로 지은 실제 도시 (실제 문장)
역할: 실제 개체와 사실의 연결망
중심 질문: 실제로 무엇이 어떻게 연결되어 있는가?
성격: 실제 데이터 네트워크 (Data Network)

TBox (Terminological Box)

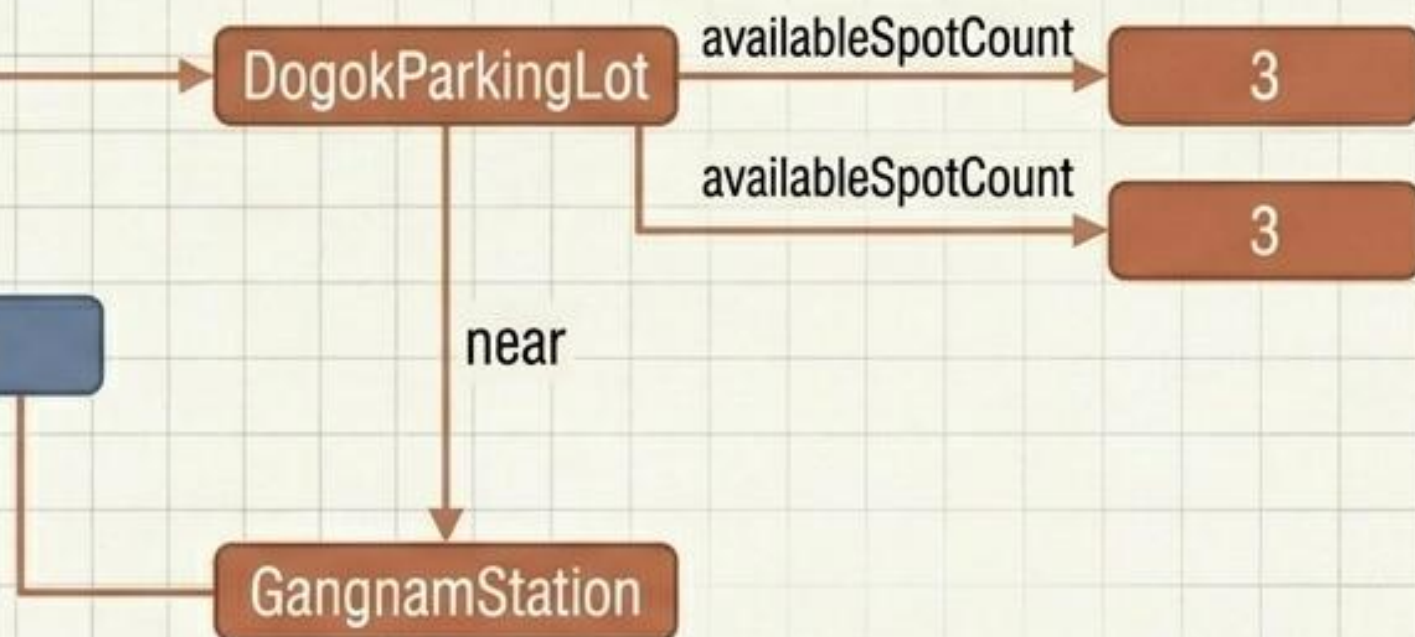
개념, 클래스, 관계, 제약을 정의하는 지식의 설계도.



- 특징: 온톨로지의 중심, 항상 참이어야 하는 공리(Axiom).
- 예시:
 - * ParkingLot은 Facility의 하위 클래스이다.
 - * hasChargingStation의 domain은 ParkingLot이다.
 - * hasChargingStation의 range는 ChargingStation이다.
 - * availableSpotCount의 값은 integer이다.

ABox (Assertional Box)

실제 개체(Individual)와 현실 세계의 사실을 기록하는 데이터.



- 특징: 현실 세계의 기록, TBox의 제약을 따르는 지식그래프의 인스턴스.
- 예시:
 - * GangnamStation은 SubwayStation이다.
 - * DogokParkingLot은 ParkingLot이다.
 - * DogokParkingLot은 GangnamStation 근처에 있다.
 - * DogokParkingLot의 availableSpotCount는 3이다.

시맨틱 AI 기술 스택 (The Semantic Tech Stack)

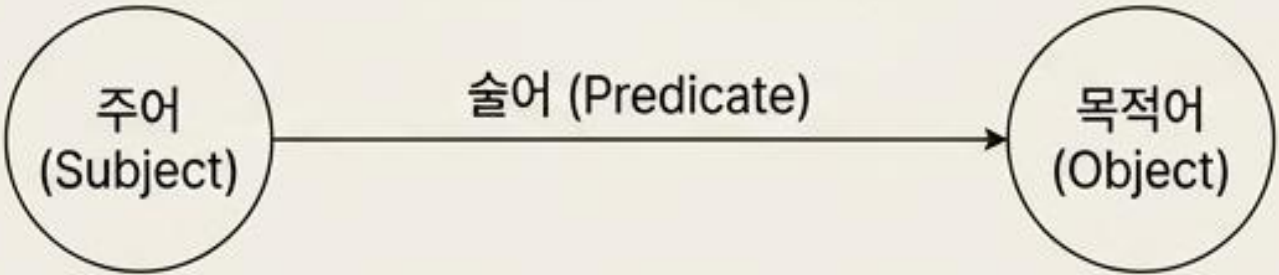


논리와 데이터의 기초: FOL과 RDF

FOL (1차 논리): 복잡한 지식의 논리적 표현

$$\forall x \forall n (\text{Place}(x) \wedge \text{avgPopulation}(x, n) \wedge n \geq 80000 \rightarrow \text{CrowdedPlace}(x))$$

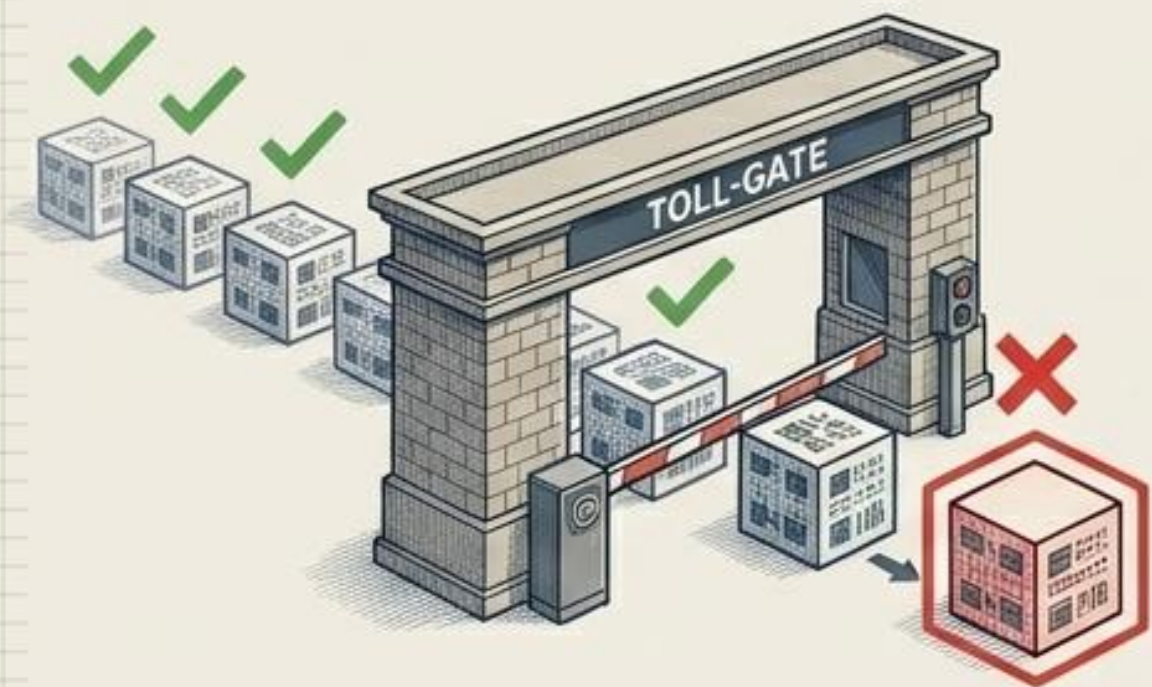
RDF (Resource Description Framework): 데이터를 주어(Subject)-술어(Predicate)-목적어(Object)로 연결하는 웹의 기본 데이터 모델.



RDF vs OWL: 데이터와 의미의 차이

	RDF	OWL (Web Ontology Language)
역할	사실을 그래프로 표현	개념, 관계, 제약, 추론 규칙 표현
기본 단위	트리플 (Triple)	클래스, 속성, 제약, 공리 (Axiom)
예시	DogokParkingLot near GangnamStation	ParkingLot은 Facility의 하위 클래스
수준	데이터 모델 (그릇)	온톨로지 언어 (의미)

SHACL: 데이터 품질의 문지기



RDF 데이터가 정해진 품질 규칙을 지키는지 검증 (예: ParkingLot은 availableSpotCount를 반드시 가져야 하며, 0 이상의 정수여야 함). 잘못된 데이터의 유입을 막아 LLM의 환각을 원천 차단.

OWL vs SHACL: 추론과 검증의 차이

구분	추론 (OWL)	검증 (SHACL)
기본 가정	개방 세계 가정 (모르는 것은 아직 모르는 것)	폐쇄 세계 가정 (필수값이 없으면 오류)
결과	새로운 지식 도출 (Inference)	검증 오류 반환 (Validation Error)

SPARQL: 의미를 탐색하는 질의어

SQL이 테이블 구조를 조회한다면, SPARQL은 그래프의 의미 관계를 따라 데이터를 조회합니다. 검증(SHACL)과 추론(OWL)이 끝난 완벽한 지식그래프를 탐색하는 핵심 도구입니다.

TTL (Turtle) 해부학: 가장 직관적인 RDF 문법

(TTL은 문법/파일 형식, RDF는 데이터 모델, OWL은 의미 언어)

```
@prefix ex: <http://example.org/> .
@prefix city: <https://data.mycompany.com/ontology/city#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

<https://data.mycompany.com/ontology/my-city>
  a owl:Ontology ;
  owl:imports <https://external.org/some-ontology> .
```

단순 축약 선언 (NOT Import)

@prefix는 외부 TBox를 가져오는 기능이 아닙니다. 파서는 이 URL에 접속하지 않으며, 단지 긴 URI를 짧게 쓰기 위한 이름표입니다.

실무용 URI

튜토리얼용 example.org 대신 회사가 관리하는 고유 URI를 사용해야 합니다.

표준 Prefix

rdf:, rdfs:, owl:, xsd:는 전 세계 표준이므로 절대 임의로 변경하면 안 됩니다.

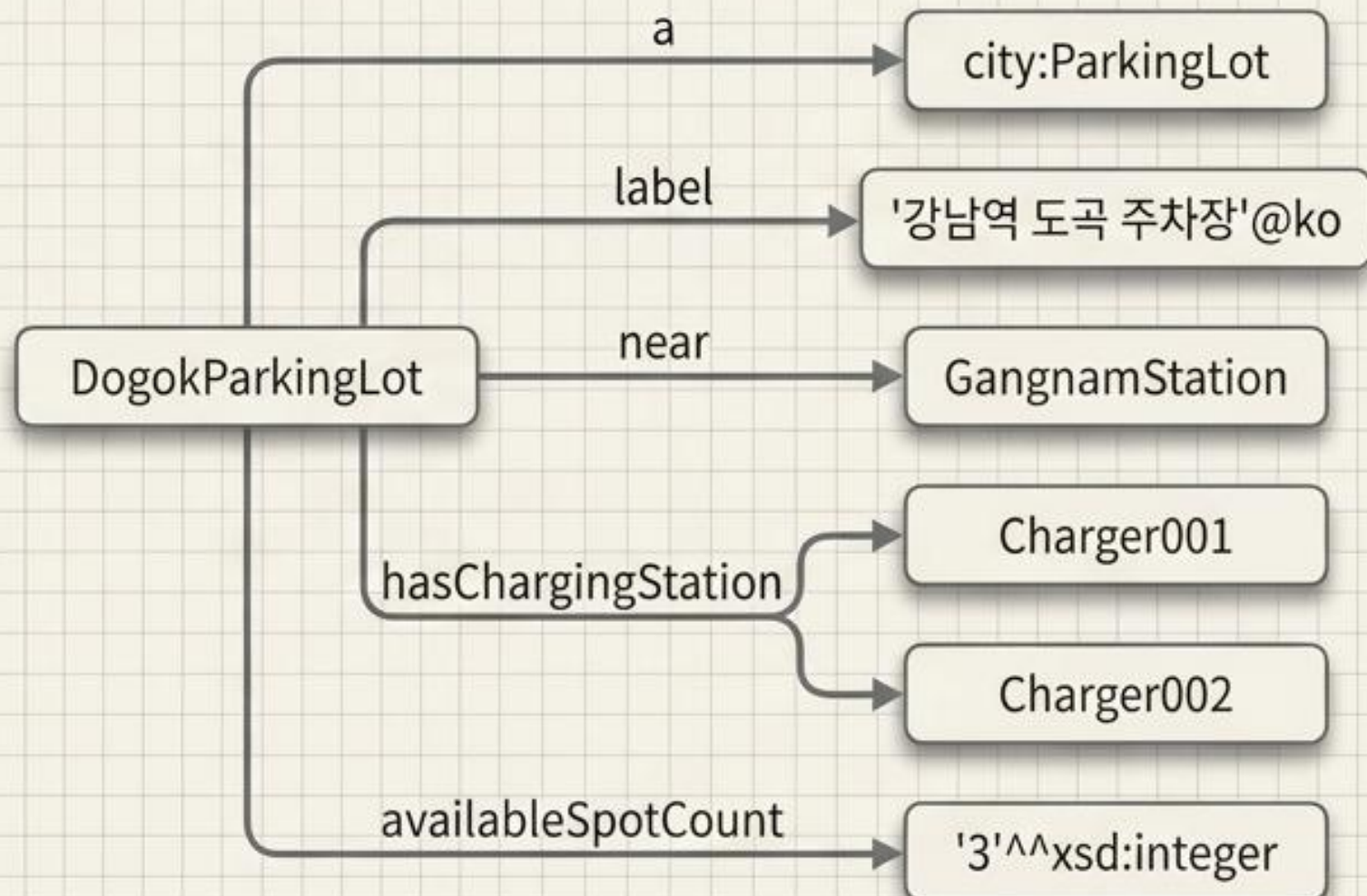
실제 외부 포함

외부 온톨로지를 논리적으로 가져오려면 owl:imports를 사용해야 합니다.

TTL 핵심 문법 cheatsheet

. (마침표)	하나의 트리플(주어-술어-목적어) 종료.
; (세미콜론)	주어를 유지한 채 새로운 술어-목적어 추가. (주어 반복 방지)
, (쉼표)	같은 주어와 술어에 여러 목적어 추가.
a	rdf:type의 축약. 'A는 B 타입이다'
'문자열'@ko	언어 태그가 붙은 리터럴 값.
'3'^^xsd:integer	데이터 타입이 지정된 값.

```
res:DogokParkingLot a city:ParkingLot ;
  rdfs:label "강남역 도곡 주차장"@ko ;
  city:near res:GangnamStation ;
  city:hasChargingStation res:Charger001, res:Charger002 ;
  city:availableSpotCount "3"^^xsd:integer .
```



OWL의 다형성: 같은 의미, 다른 문법 (The Rosetta Stone)

OWL 온톨로지는 논리적 의미일 뿐, 다양한 직렬화 포맷(RDF/XML, TTL, JSON-LD 등)으로 표현할 수 있습니다.

포맷 1: TTL (Turtle)

(가장 널리 쓰이는 RDF 기반 표현)

```
city:ParkingLot a
  owl:Class ;
  rdfs:subClassOf
    city:Facility .

city:hasChargingStation
  rdfs:domain city:ParkingLot ;
  rdfs:range
    city:ChargingStation .
```

포맷 2: Manchester Syntax

(사람이 읽기 쉽고 Protégé에서 주로 사용)

```
Class: ParkingLot
  SubClassOf: Facility

ObjectProperty: hasChargingStation
  Domain: ParkingLot
  Range: ChargingStation
```

포맷 3: OWL Functional Syntax

(논리 구조를 가장 엄밀하게 표현)

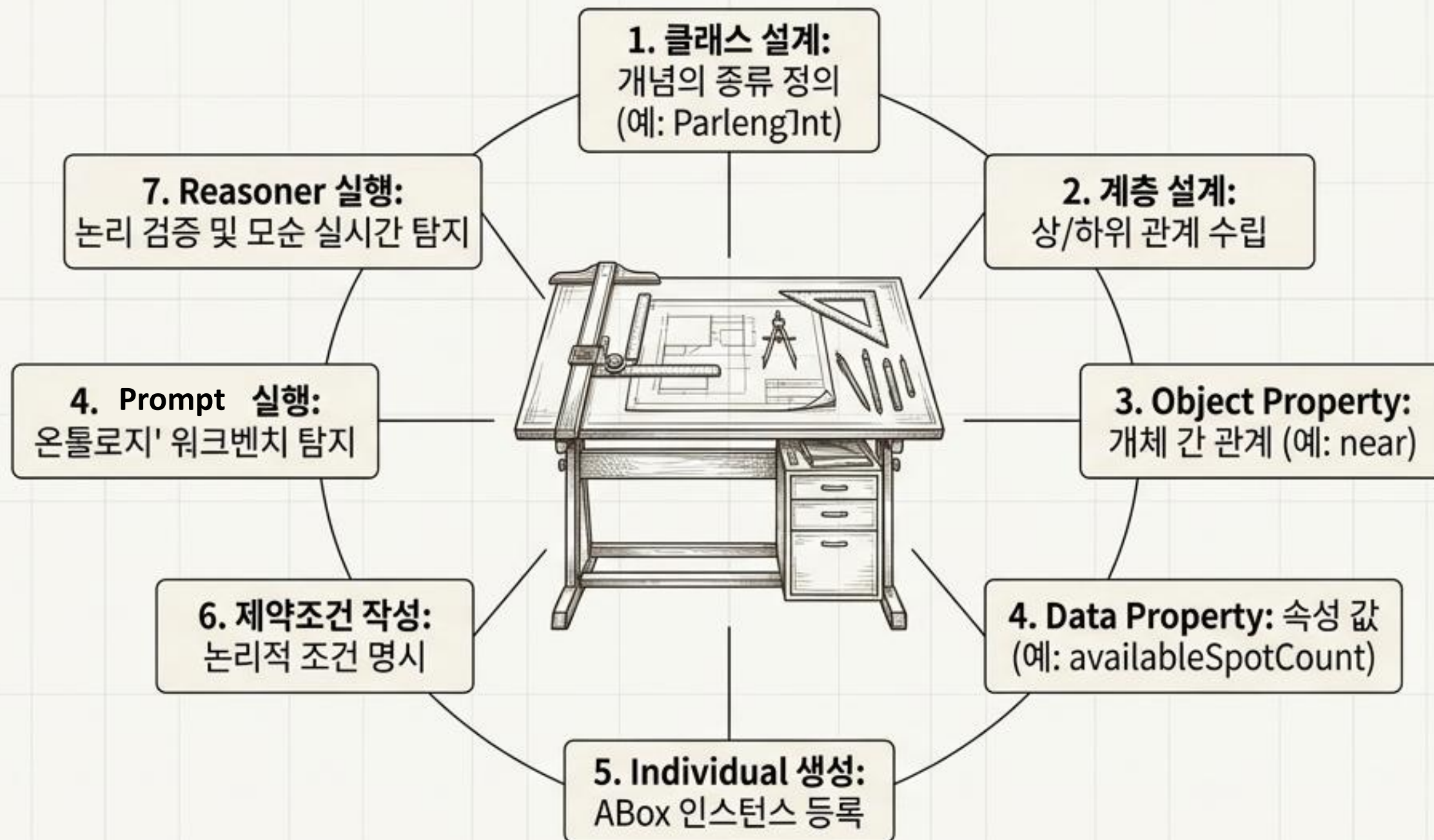
```
Declaration(Class(:ParkingLot))

SubClassOf(:ParkingLot
  :Facility)

ObjectPropertyDomain
  (:hasChargingStation
    :ParkingLot)
```

Protégé: 온톨로지 설계자의 워크벤치

Protégé는 단순한 다이어그램 그리기 도구가 아닙니다.
TBox를 설계하고 OWL Reasoner로 실시간 검증하는 통합 개발 환경(IDE)입니다.



OWL Reasoner: 숨겨진 진실을 찾는 논리 엔진

새로운 데이터를 생성하는 것이 아니라, 정의된 TBox를 기반으로 암묵적 사실(Inferred facts)을 도출합니다.

1. 클래스 계층 추론 Facility 안에 자동으로 EVChargingParkingLot 분류.	2. 인스턴스 타입 도출 충전기가 있는 주차장을 자동으로 EVChargingParkingLot 타입으로 지정.
3. 모순 검사 동시에 존재할 수 없는 주거용/상업용 건물 속성 탐지.	4. 속성 특성 활용 대칭(Symmetry) 및 전이(Transitivity) 관계 추론 (A near B $\rightarrow$ B near A).

개방 세계 가정 (Open World Assumption)

데이터에 없다고 해서 거짓은 아니다.

주차장의 충전기 데이터가 누락되었다면, Reasoner는 '충전기가 없다'고 단정하지 않고 '아직 모른다'고 판단합니다. 이는 단순 누락을 오류로 처리하는 SHACL(폐쇄 세계 가정)과 가장 큰 차이점입니다.

OWL Reasoner 도구 생태계: 연구용부터 엔터프라이즈까지

실무에서는 완벽한 OWL DL 추론(개발/검증)과 대용량 제한적 추론(운영)을 구분해야 합니다.

오픈소스 및 정통 Description Logic 추론기 (설계/검증용)

- **HermiT**: OWL 2 DL 추론, Protégé의 기본 탑재 엔진
- **Pellet / Openllet**: 설명(Explanation) 기능과 Java 연동이 뛰어남
- **FaCT++ / ELK / Konclude**: 대형 온톨로지 성능 실험, 특화 프로파일(OWL 2 EL) 처리



지식그래프 DB (Triple Store) 내장 추론 엔진 (운영/대규모 데이터용)

- **GraphDB / Stardog / AllegroGraph**: 대규모 RDF 저장, SPARQL 질의, RDFS/OWL 2 RL/OWL-Horst 수준의 운영 환경 최적화 추론 제공
- **RDFox**: 고성능 Rule-based 인메모리 추론

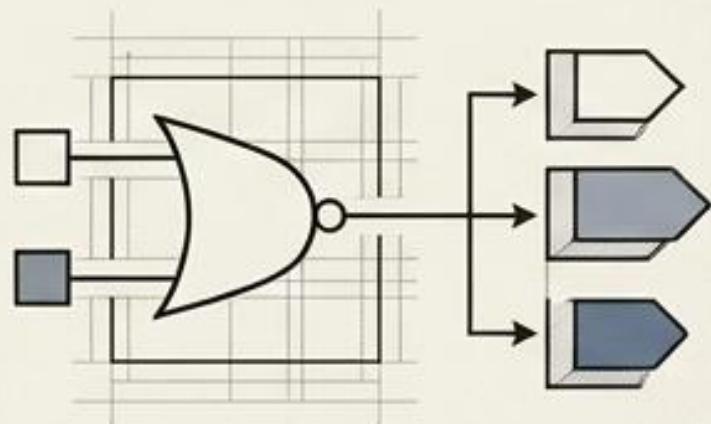


시너지의 완성: Reasoner vs SPARQL vs LLM

세 기술은 경쟁 관계가 아닙니다. 논리적 확장, 정밀한 추출, 자연어 생성의 완벽한 보완 관계입니다.

OWL Reasoner (논리 엔진)

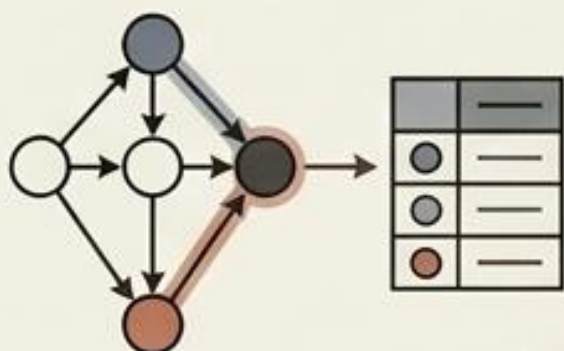
- 역할: 지식 자체를 논리적으로 확장 (Inferred Facts)
- 질문: 무엇이 논리적으로 따라오는가?



Inferred Facts

SPARQL (질의 언어)

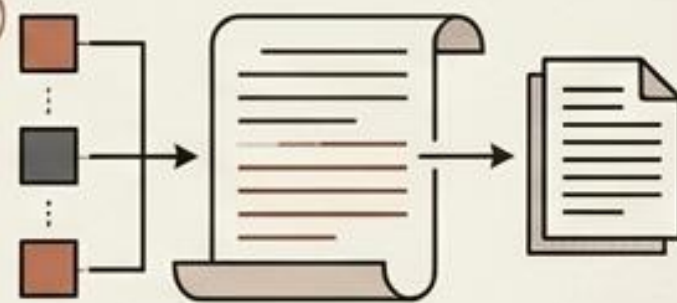
- 역할: 확장된 지식에서 조건을 만족하는 사실 도출 (Query Result)
- 질문: 조건을 완벽히 만족하는 데이터는 무엇인가?



Query Result

LLM / RAG (설명 엔진)

- 역할: 도출된 사실을 바탕으로 자연어 답변과 근거 제시 (Answer & Explanation)
- 질문: 사용자에게 어떻게 가장 잘 설명할 것인가?

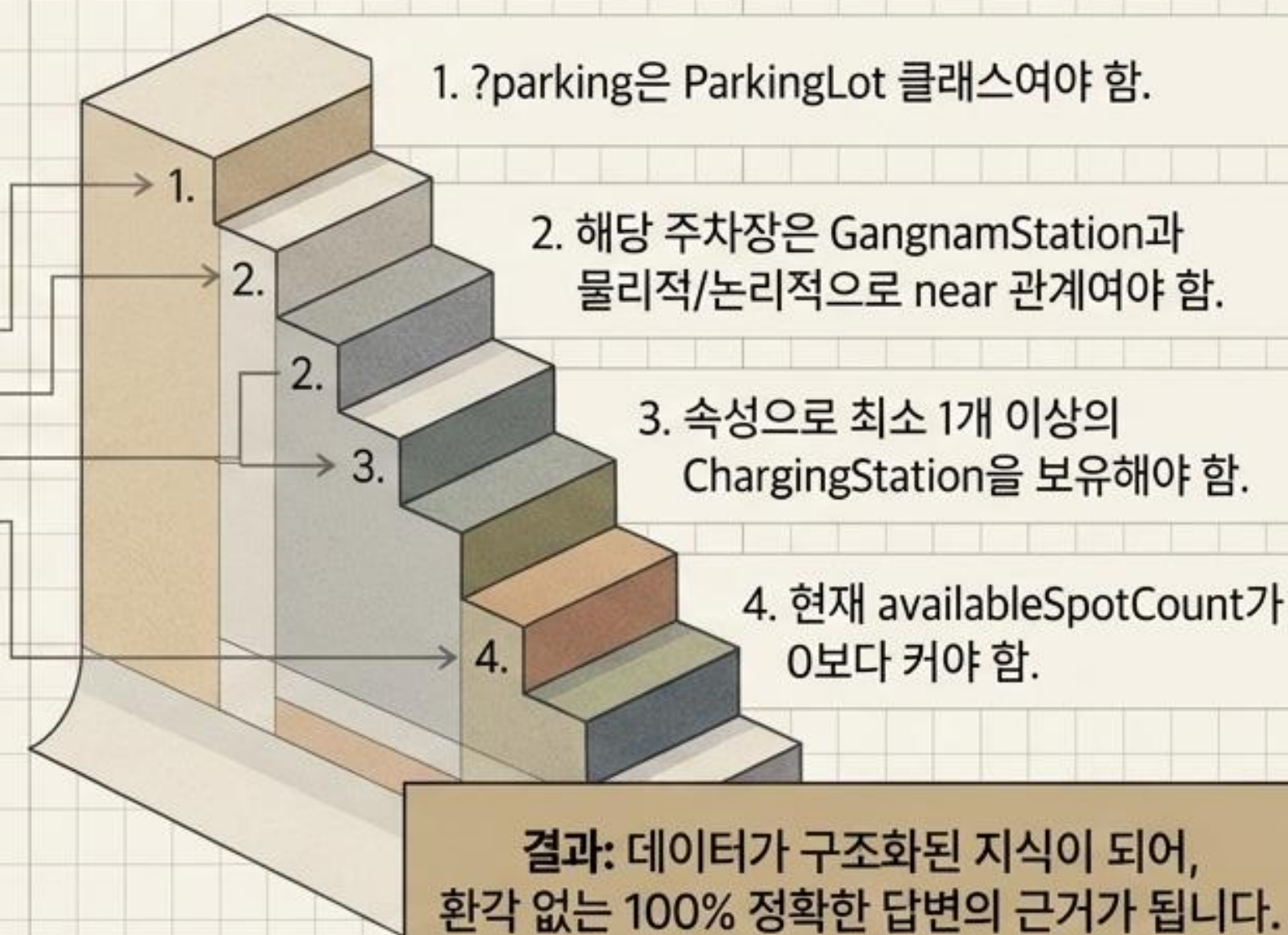


Answer & Explanation

키워드 검색이 아닌 '의미 구조' 질의

단순 검색은 텍스트 내 문자열을 찾지만, SPARQL은 '의미 조건'을 모두 만족하는 엔티티를 찾습니다.

```
SELECT ?parking ?count WHERE {  
  ?parking a city:ParkingLot ;  
  city:near res:GangnamStation ;  
  city:hasChargingStation ?charger ;  
  city:availableSpotCount ?count .  
  
  FILTER(?count > 0)  
}
```



실무 구축 파이프라인 Phase 1: 목표 정의부터 온톨로지 설계까지

데이터가 아닌 '질문'에서 출발하여 의미 구조를 모델링합니다.

Step 1. 목표 질문 정의

'정체 구간 근처의 충전 가능 주차장은?'
처럼 구체적 비즈니스 질문 도출.

1

Step 3. 핵심 클래스 도출

독립적 개체(주차장, 지하철역,
충전기 등)를 명사 중심으로 도출.

3

Step 5. Protégé 설계

TBox 작성 및 클래스
계층/규칙 구조화.

5

Step 2. 데이터 탐색

API, JSON 등의 원본 데이터
형태 및 누락된 의미 파악.

2

Step 4. 프로퍼티 정의

개체 간 관계(Object Property)와
개체-값 관계(Datatype Property) 분리.

4

Step 6. 논리 검증

OWL Reasoner로 모순
및 정합성 테스트.

6

실무 구축 파이프라인 Phase 2: 데이터 적재부터 LLM/RAG 연동까지

설계된 구조에 데이터를 채우고 사용자 서비스로 연결합니다.

Step 7. RDF/TTL 저장

실제 데이터를 ABox
트리플 형식으로 변환.

7

Step 9. Triple Store 적재

GraphDB, Stardog 등
지식그래프 DB에 구축.

9

Step 11. LLM/RAG 연결

SPARQL 조회 결과를 LLM의
프롬프트 컨텍스트로 주입.

11

Step 8. SHACL 검증

데이터 적재 전, 필수값
및 타입 오류 필터링.

8

Step 10. SPARQL 질의

목표 질문에 기반한
의미 질의 템플릿 실행.

10

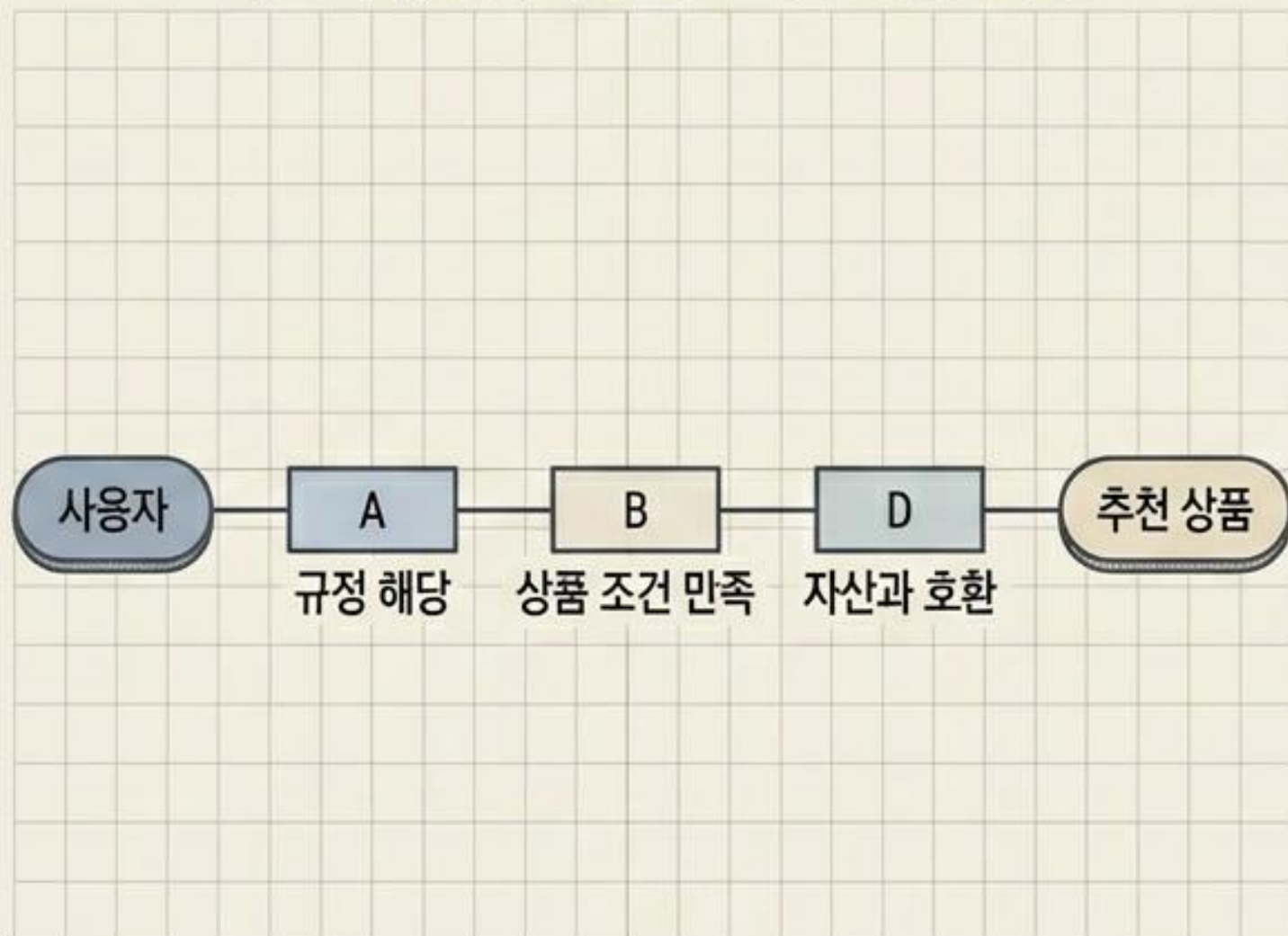
Step 12. 서비스 확장

환각 없는 추천, 질의응답,
자동화 챗봇으로 배포.

12

Use Case 1: 설명 가능한 고도화 추천 시스템

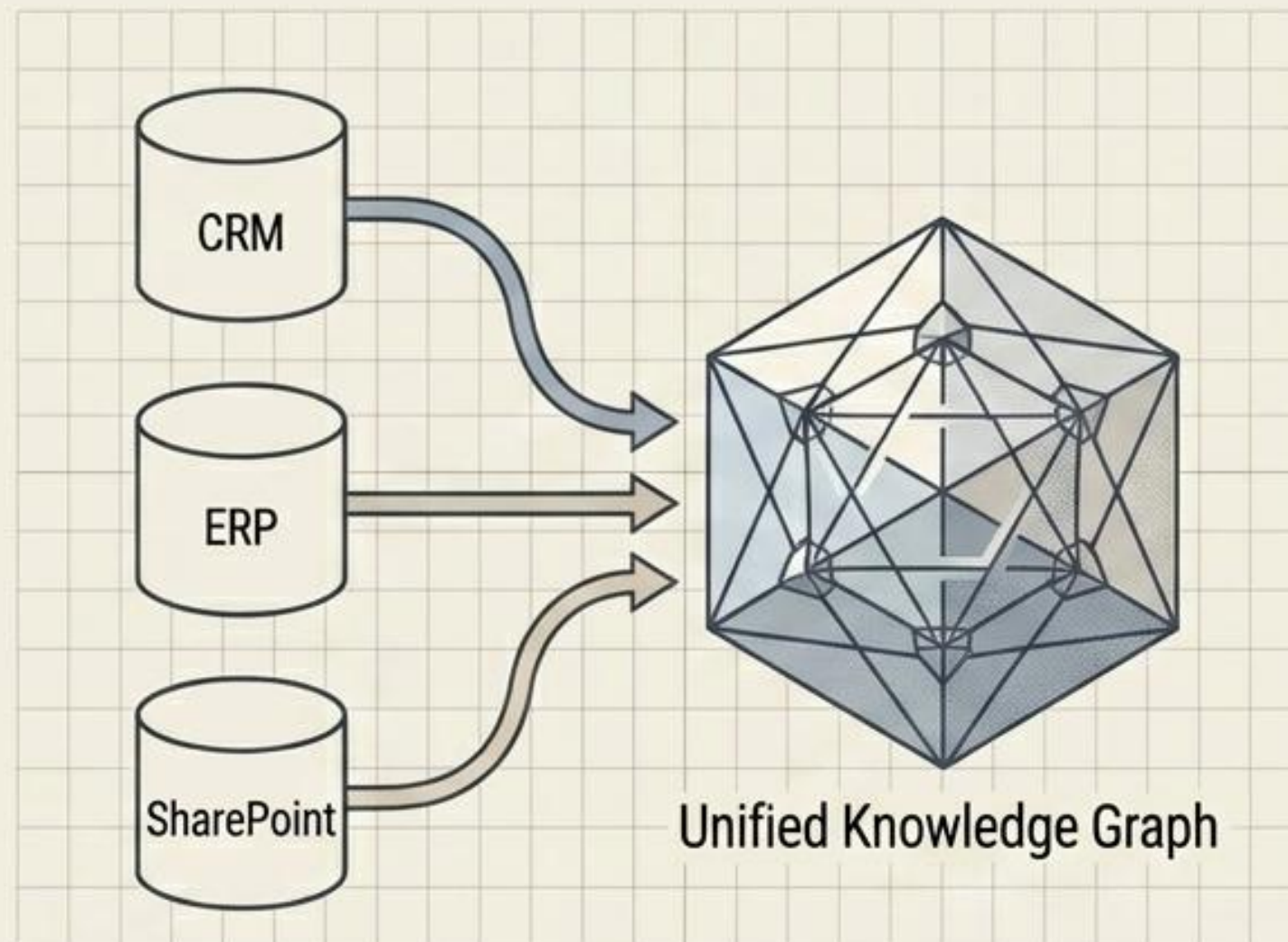
단순한 '유사도' 기반 추천을 넘어 업무 규칙, 도메인 제약, 실시간 컨텍스트를 반영한 추천.



- 로직 추적: 고객이 A 규정에 해당하고 B 상품 조건을 만족하며 보유 자산과 호환되므로 추천합니다.
- 효과: 지식그래프의 트리플 경로 추적을 통한 100% 투명한 추천 사유 제공.

Use Case 2: 기업 내 데이터 사일로 통합

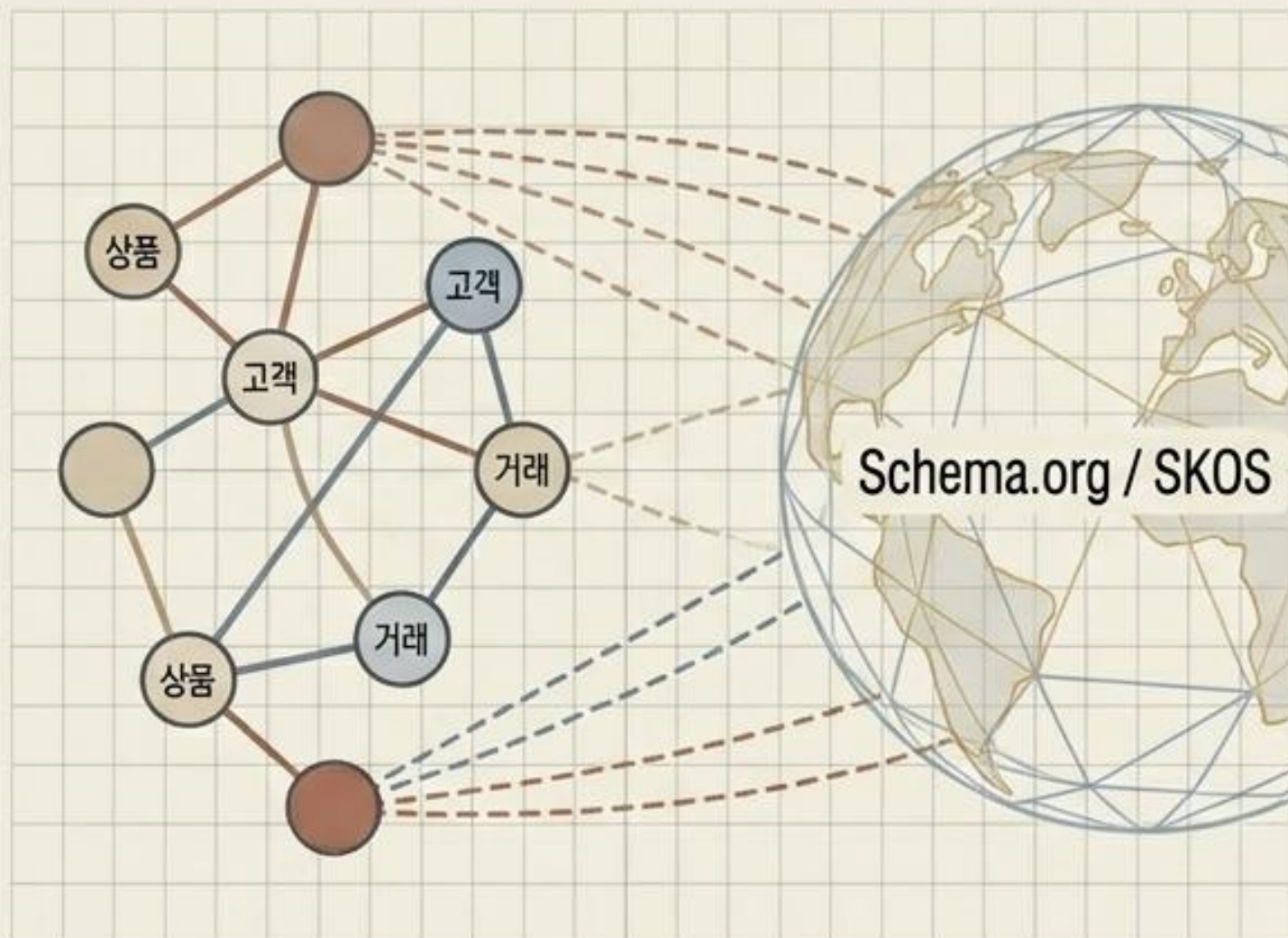
파편화된 엔터프라이즈 데이터를 의미적으로 결합.



- 구조: CRM(고객) + ERP(자산) + SharePoint(문서) + VOC(문의 로그).
- 효과: 최근 성능 이슈를 문의한 고객 중 A 제품을 쓰는 사람에게 필요한 기술 문서는? 같은 복합 도메인 횡단 질의 가능.

Use Case 3: 외부 표준 지식망과의 연결

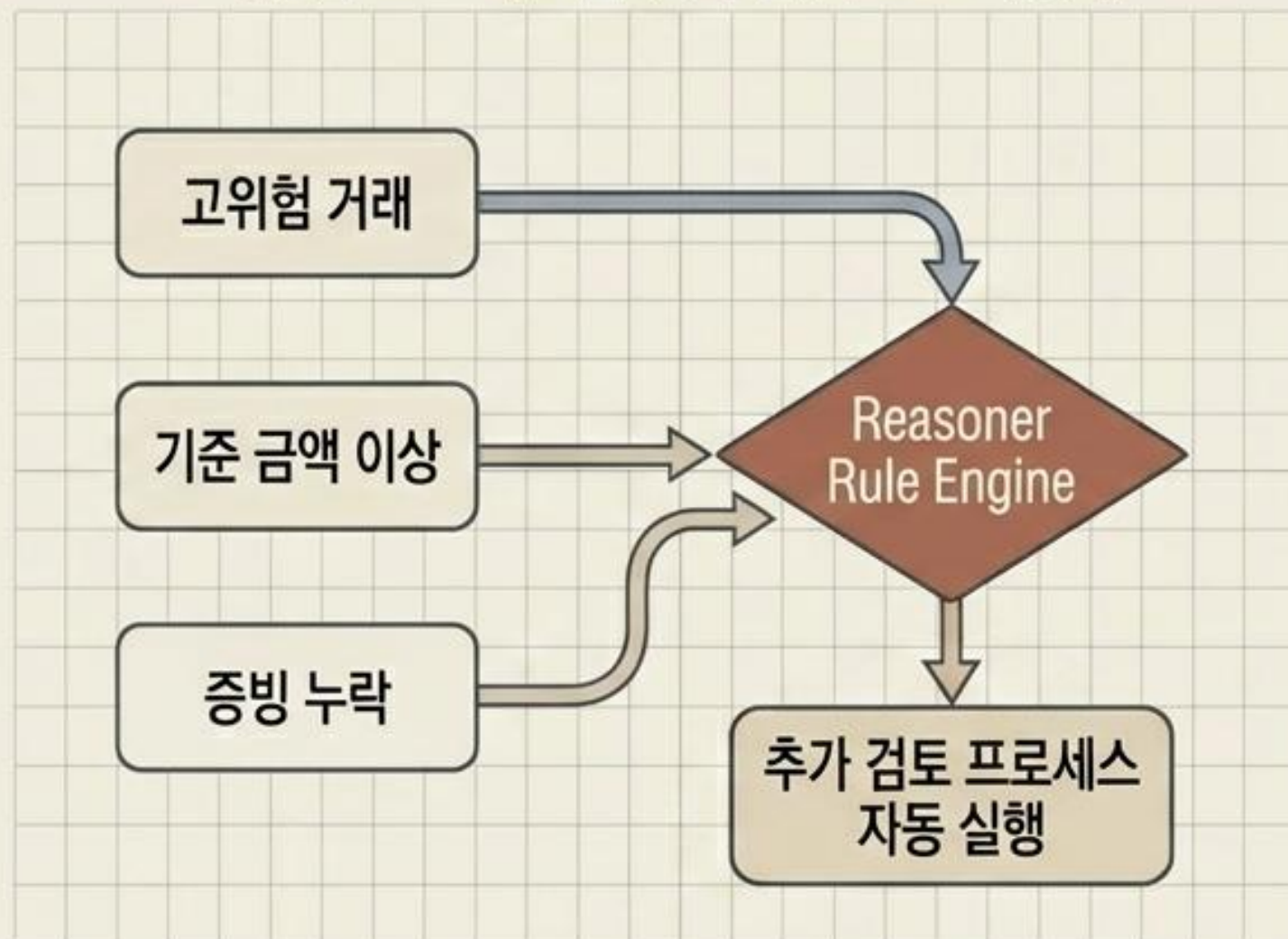
내부 지식을 글로벌 표준 온톨로지와 매핑하여 확장.



- 연결점: Schema.org, SKOS, GeoSPARQL, Dublin Core 등.
- 효과: 내부 상품 코드와 국제 분류 통합, 지리 정보(GeoSPARQL) 결합을 통한 공간 분석 및 규제 데이터 실시간 대응 가능.

Use Case 4: 정책 자동화 및 BPM

업무 규칙을 지식그래프로 표현하여 복잡한 프로세스를 논리 엔진으로 자동화.

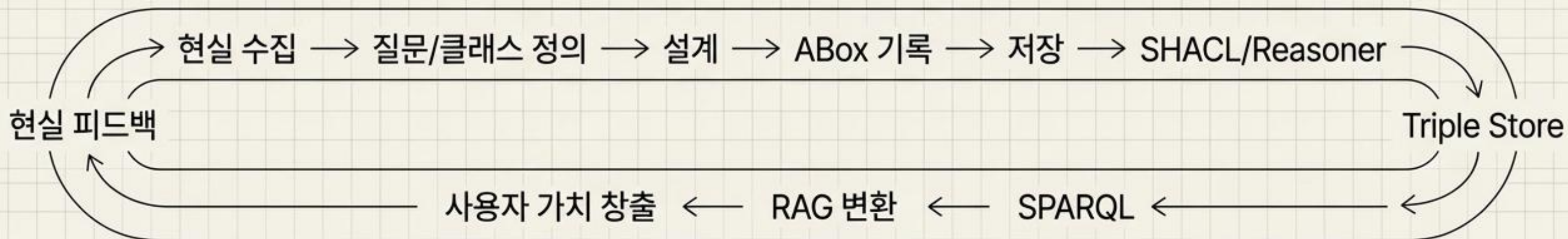


- 동작 방식: 조건 충족 시, Reasoner와 Rule Engine이 추가 검토 프로세스를 자동 실행.
- 효과: 단순 LLM 기반 판단보다 훨씬 강력한 감사 추적성 (Auditability)과 무결성 확보.

마스터 플랜 치트시트:

Ontology (의미 설계도)	KG (지식 네트워크)	TBox/ABox (구조/데이터)	FOL (논리 기초)
RDF (트리플 모델)	TTL (RDF 문법)	OWL (추론 언어)	SHACL (품질 검증)
SPARQL (그래프 질의)	Protégé (설계 도구)	Reasoner (추론 엔진)	LLM (언어 설명기)

13-Step 생태계:



온톨로지는 LLM을 대체하지 않습니다. 온톨로지는 LLM이 환각 없이 신뢰할 수 있고, 일관되며, 설명 가능한 능력을 갖추게 만드는 필수적인 '의미 기반 인프라'입니다.